

The 8086 Microprocessor~

ARCHITECTURE OF 8086.

→ 16 bit microprocessor (MP)

→ It contains 20 bit address bus
can address upto 1 MB memory.

bus → set of wires.

If 2 bit address bus the we
can address 4 location. (2^2)

8 bytes = 1 bit

→ This contains 16 bit data bus

Data bus: set of wires which carries
data.

→ This contains set of registers

*. General purpose.

*. special purpose.

→ For storage purpose registers used.

→ Group of flipflop is register.

→ It supports segmented memory
organization. [for easy access]

→ Memory divided into 4 segment
each will be of 64 kBytes totally 1 MB.

Architecture of 8086 can be categorized into 2 units.

1. Bus interface unit (BIU)

2. Execution unit (EU) →

BIU → It fetch the information (instruction)
→ decoding the instruction.
→ directs the data transfer to / from memory unit.

EU

→ It will have set of register.

→ It has 4 General purpose register
(AX, BX, CX, DX) all are 16 bit registers.

→ each 16 bit registers are used as 2 8 bit registers.

higher half lower half

AX	AH	AL
BX	BH	BL
CX	CH	CL
DX	DH	DL
	SP	
	BP	
	SI	
	DI	

AL → lower byte register.

AH →

AX → higher & lower order byte

special registers : 16 bit register.

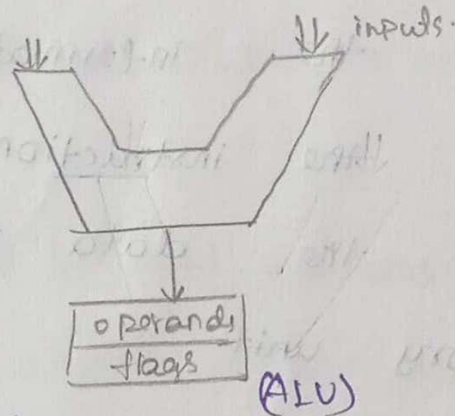
SP $\rightarrow$ stack pointer. For stack operation

BP $\rightarrow$ Base pointer. used to store the offset address

SI $\rightarrow$ source Index

DI $\rightarrow$ destination Index

} For string op



ALU



arithmetic & logical unit.

$\rightarrow$ It has two inputs.

Flags $\rightarrow$ has status information.

* 9 Flags are there:

① CF $\rightarrow$ carry flag.

② PF $\rightarrow$ Parity flag.

$\Rightarrow$ when parity is even, it is set to 1
(no. of ones)

③ AF $\rightarrow$ Auxiliary carry flag.

$\Rightarrow$ when there is carry from lower

nibble to up nibble.

(1 bit)

④ ZF $\rightarrow$ zero flag.

$\Rightarrow$ If the result is 0, then this bit will be set to 1.

⑤ SF $\rightarrow$ sign flag

$\Rightarrow$ If result is -ve, the SF will be 1.

⑥ OF $\rightarrow$ overflow flag

when the result can not be accommodated (not stored) then it is set to 1.

⑦ TF $\rightarrow$ trap flag

$\Rightarrow$ single step programming.

single step - only one instruction performed at a time.

$\Rightarrow$ It is set to 1, when it performs in single step mode.

⑧ IF $\rightarrow$ Interrupt flag

⑨ DF $\rightarrow$ direction flag

$\Rightarrow$ to indicate the direction in string operation.

4 segments registers

CS $\rightarrow$ code segment

* program instruction

DS $\rightarrow$ data segment

* all the data available here

SS $\rightarrow$ stack segment

* stack operations

ES $\rightarrow$ Extra segment.

*. additional data segment.

IP $\rightarrow$ Instruction pointer. (it has offset)

*. It stores the address of next instruction to be fetched.

$\Rightarrow$ to store 16 bit base - CS, SS, DS, ES

base address - start address.

CS - It has the base address of code segment.

Similarly for DS, SS, ES.

eg:

$$\begin{array}{r} \text{CS : } 4321 \text{ H} \times 10 \\ \text{IP : } 1000 \text{ H} \\ \hline 44210 \end{array}$$

Formula : $\text{CS} \times 10 \text{ H} + \text{IP}$

$\Downarrow$

to find physical address

here we want access

to complete 1 instruction. (require 3 cycles)

1. Instruction fetch.

2. Instruction decode.

3. Execute.

Diagram

18/12/25

ADDRESSING MODES OF 8086.

Addressing modes refers to the different ways by which the source operand is denoted in the an instruction. is called addressing mode.

MOV $\rightarrow$ copy a byte from source to destination.

(or)
word.

① Immediate Addressing mode.

eg: MOV AX, 42H.

D.

S.

Before

AX

After

42H

AX.

H $\rightarrow$ Hexadecimal

Here source operand is directly given in the instruction.

If the source operand is directly given in the instruction is called immediate addressing mode.

② Register Addressing Mode:

If source operand is given in the register is called register addressing mode.

eg: MOV AX, BX

Before

AX

-
42H

BX

42H

source
After

AX

42H

BX

42H

42H (source) is copied to AX.

(destination)

③. Direct Addressing Mode.

If the address of the source operand is directly given in the instruction is called direct addressing mode.

without $[]$ $\rightarrow$ it is data.

within $[]$ $\rightarrow$ it is address.

eg: `MOV AX, [1200H]`

Before.

AX

1200H 42

After.

AX 42

↓
address of the data.

④. Register Indirect Addressing Mode.

If address of the data is given in the register is called Register Indirect Addressing Mode.

eg: `MOV AX, [BX]`

Before.

AX

BX 1200H

1200H 42H

After.

AX 42H

⑤. Based Addressing Mode:

It should be base register (BX (or) BP) only

In Based Addressing Mode, offset

address is given by ^{sum} some of
content of BX / BP register with
8 bit (or) 16 bit displacement

eg: `MOV AX, [BX + 12H]`

Before.

After.

8 (or) 16 bit
displacement

AX

AX 12

BX 1200H

$$BX + 12 = 1212$$

1212H 12

⑥. Index Addressing mode.

The offset address is computed
by summing SI & DI register is
called Index Addressing mode.

eg: `MOV AX, [SI + 12H]`

Before.

After.

AX -

AX 12

SI 1200H

$$SI + 12H = 1212H \quad \text{1212H} \quad \text{12}$$

⑦. Based Indexed Addressing mode.

offset address is computed by summing SI, DI register, BX/BP register and 8 or 16 bit register displacement.

eg: `MOV AX, [SI + BP + 12H]`.

Example:

1. `ADD AX, BX` - Register Addressing mode.
2. `MOV SI, [1200H]` - Direct Addressing mode.
3. `SUB AX, 12H` - Immediate addressing mode.

INSTRUCTION SETS OF 8086.

1. Data Transfer Instruction
2. Arithmetic Instructions.
3. BH manipulation Instructions.
4. String Instructions.
5. Program Execution Transfer Instruction
6. Processor control Instruction.
7. Iteration control Instruction.
8. Interrupt Instructions.

①

DATA TRANSFER INSTRUCTIONS.

* MOV - copy a byte / word from source to destination.

eg: MOV AX, 42H

MOV AL, 42H

* PUSH - It will push a word on to the top of the stack

eg: PUSH 1200H

address.

The data in the address is pushed in top of stack
(1200H)

* POP - It will get a word from the top of the stack

eg: POP 1200H

(word) data in top of stack is moved to the address 1200H.

* PUSHF - used to put all registers into the stack.

* POPF - get word from stack to all registers.

* XCHG - exchange data from two locations.

*. XLAT - Translate a byte in ~~acc~~ AL using a table in memory.

*. IN / OUT - (data from peripheral to micro processor in IN instruction).

Condition: we shouldn't use any other instruction. only accumulator eg. is used. (x)

IN - send a byte from port to acc (accumulator) eg: IN AL, 81H port address
OUT - send a byte from acc to port.

eg: OUT AL, 81H.

acc. data is moved to port address 81H.

Instructions to transfer address

*. LEA → load effective address into provided register.

*. LDS → load data segment register from memory.

*. LES → load extra segment register from memory.

Instructions to transfer flag register.

*. LAHF → load AH with low byte of flag register.

16 bit register

*. SAHF $\rightarrow$ It will store AH content to lower byte of flag register.

②. ARITHMETIC INSTRUCTIONS: $\Rightarrow$ one of operand should ~~add~~ in Ax & it's result is stored in Ax.

*. ADD

*. ADC - add with carry

$\Rightarrow$ carry bit ~~add~~ in flag reg

*. INC - Increment

*. AAA - ASCII Adjust after addition.

*. DAA - Decimal adjust after addition.

Based on subtraction.

*. SUB.

$Ax - Bx$ - result stored in Ax.

*. SBB - subtract with borrow.

*. DEC - decrement

DEC Ax $\rightarrow$ decrement by -1 in Ax.

*. NEG - ~~to~~ 2's complement.

*. CMP - compare

$Ax - Bx$ = but not result is updated in Ax.

flag register only updated.

*. AAS - ASCII adjust after subtraction.

*. DAS - decimal adjust after subtraction.

In multiplication one register only used to represent.

*. MUL - multiplication. (unsigned)

eg: MUL BX then $(AX) \times (BX) = \text{result stored in } AX.$

If AX is 16 bit, BX is 16 bit = result is 32 bit
then lower order word 16 bit store in AX,
& higher order word 16 bit store in DX.

*. IMUL - signed multiplication.

*. DIV - unsigned division.

eg: DIV BX. then meaning is AX/BX , after $\div$
Quotient stored in AX, remainder stored in DX.

$$\begin{array}{r} Q \\ BX \overline{) AX} \end{array}$$

*. IDIV - signed division.

Remainder.

③. BIT MANIPULATION INSTRUCTION:

i). Logical Instruction.

• AND

eg: AND AX, BX.

AX = 1001, BX = 0111

AX = 0001.

• OR

• XOR

• NOT

• TEST - used to add operands,

to update the flag register without affecting the operands.

ii). shift instructions.

*. SHL / SAL - shift left / shift arithmetic left

*. SHR - shift right.

*. SAR - shift arithmetic Right.

eg: SHL AX, 01H $\Rightarrow$ 1 bit shift left

AX = 1001.

then after shifting = 0010

SHR AX, 01H

AX = 1001, then after right shift = 0100

In SAR.

here old MSB is copied to NEW MSB.

AX = 1001, then after $\frac{1}{1}001$

↓ old MSB placed in New MSB.

iii). Rotate Instructions.

*. ROL - Rotate left

*. ROR - Rotate right.

*. RCR - Rotate right through carry.

*. RCL - Rotate left through carry.

eg: AX = 1001, carry 1

ROR - 0011, RCL - 1100, RCL - 1001

⑤.

PROGRAM CONTROL TRANSFER INSTRUCTIONS:

*. CALL - used to call a procedure.

*. RET - stands for Return.

- used to return from the procedure to the main program.

*. JMP - Jump.

once jump to other then it will not return to the starting. [Program control doesn't return]

*. JA - Jump if above.

JA AX, BX, when $AX > BX$ then jump to (1020 H).
when $AX < BX$ then go to next line of sequence.

Conditional

*. JB - Jump if below.

*. JE - Jump if equal.

*. JZ - Jump if zero.

*. JP - Jump if parity.

*. JO - Jump if overflow.

*. JS - Jump if sign.

⑥.

PROCESSOR CONTROL INSTRUCTIONS:

*. STC → set carry.

*. CMC → clear carry.

*. EMC → complement carry.

*. STD → set direction flag.

(decrement by 1.)

* CLO $\rightarrow$ clear direction flag, (address is increment)

* CLI $\rightarrow$ clear interrupt flag.

⑦. ITERATION CONTROL INSTRUCTIONS.

* LOOP -

initially we have to set the cx. for each loop the count will decrement. loop executes till it reach $cx=0$.

initially, $cx=5$, is not equal to loop executes.

same for $cx=4, 3, 2, 1$ loop executes $cx=0$ loop stop.

after every iteration it checks the cx value.

* LOPE - loop if equal.

it checks $cx \neq 0$, zero flag ($ZF=1$)

when both condition satisfied the iteration is stop otherwise it continues.

* LOOPNE - loop if not equal.

condition : $cx \neq 0$, $ZF=0$. until it reach this condition loop executes.

⑧. INTERRUPT CONTROL INSTRUCTIONS.

* INT - Interrupt.

* INTO - interrupt on overflow.

* IRET - interrupt return. (return to where it starts) after interruption.

interrupt currently going process is temporarily suspended.

(1)

STRING MANIPULATION INSTRUCTIONS:

* REP - repeat,
 → move data from SI to DI

* MOVSB - Move string.
 → increment by 1

it can be i) MOVSB - move string byte.

ii) MOVSW - move string word

* CMPS - compare string.

[comparing strings]

CMPSB

↳ byte.

CMPSW

↳ compare string word.

* SCAS - scan string ~~byte~~
 [Finding a byte/word in string]

SCASB - byte.

SCASW - word.

* LODS - load string

Load byte/word to AL/AX
 by content of string.

LODSB = byte.

LODSW = word.

* STOS - store string

STOSB = byte.

STOSW = word.

string ⇒ group of bytes which
 stored in a consecutive location.

eg:

MOV CX, 05H

MOV SI, 1000H
 → start address.

MOV DI, 2000H
 → destination address.

CID

REP MOVSB.

SI → source string.

DI - destination
 string.

CID - clear direction
 flag.

address incremented
 both SI & DI

parameter:

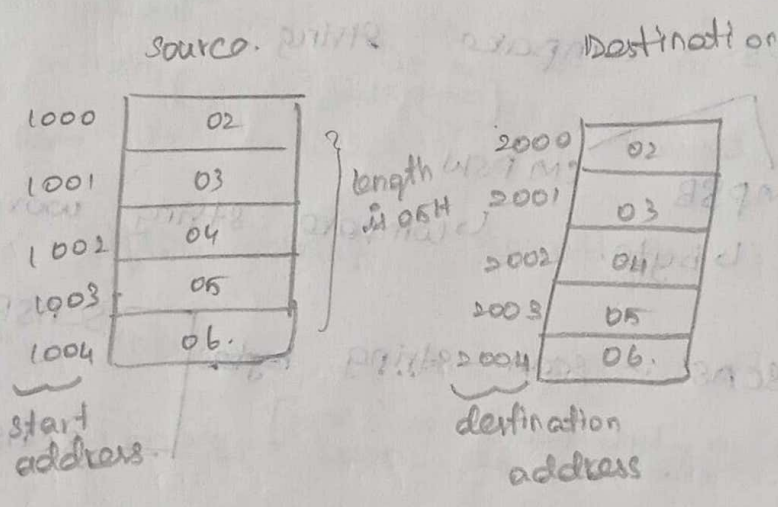
⇒ to specify length of string

⇒ to specify start address

CX - count register = to store the length of string

REP - it repeats until CX = 0.

MOVSB → it moves data in SI address to DI address.



eg:

MOV CX, 05H

MOV SI, 1000H

MOV DI, 2000H

CJD.

REPE CMPSB

REPE → Repeat if equal.

CMPSB → compare string byte.

REPE → repeats if SI = DI & CX ≠ 0

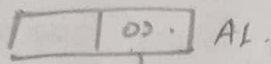
SI ≠ DI & CX = 0 it doesn't repeat

compares SI & DI (CMPSB)

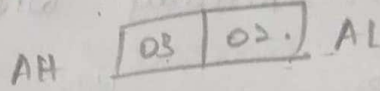
The byte in 1000 H is load to AL

eg: MOV SI, 1000 H

LODSB.



LODSW.

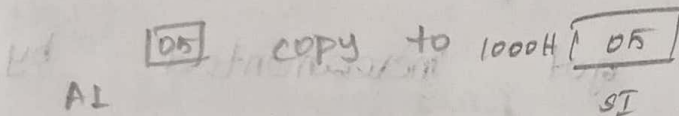


eg:

MOV SI, 1000 H.

STOSB. → accumulator to string.

The data in AL is copied to SI of given address 1000 H.



STACK:

* successive memory location used to store temporary byte (or) data.

* It works in LIFO (last in first out.) principle.

* operations.

PUSH - inserting a byte into the stack.

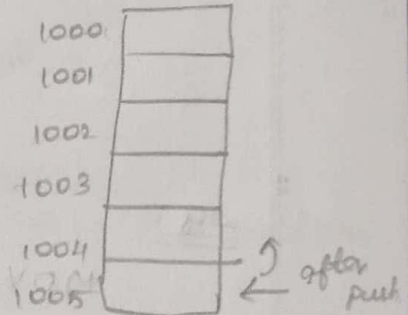
POP - removing a byte from the top of the stack.

* Stack pointer is 16 bit register. that will have address of top of the stack.

* Initially when stack is empty. then SP. ^{here we consider stack segment is 0.}

SP 1005

here push operation is done in 1005.



* after every push operation, stack pointer will be decremented by 1 (byte) (or) 2 (word).

* after every pop operation, stack pointer got incremented by 1 (For byte) (or) 2 (For word)

SP - offset address of the stack

The Real address should be in 20 bits.

* to find Real address of stack

pointer = Stack segment $\times$ 10H + offset address.

procedure - small subsection of program.

It stored only once.

PROCEDURES OF MACROS:

A group of instructions that usually performs one task is called as procedure, it is a reusable section of a software program which is stored in memory once but can be used as often as necessary.

Two types.

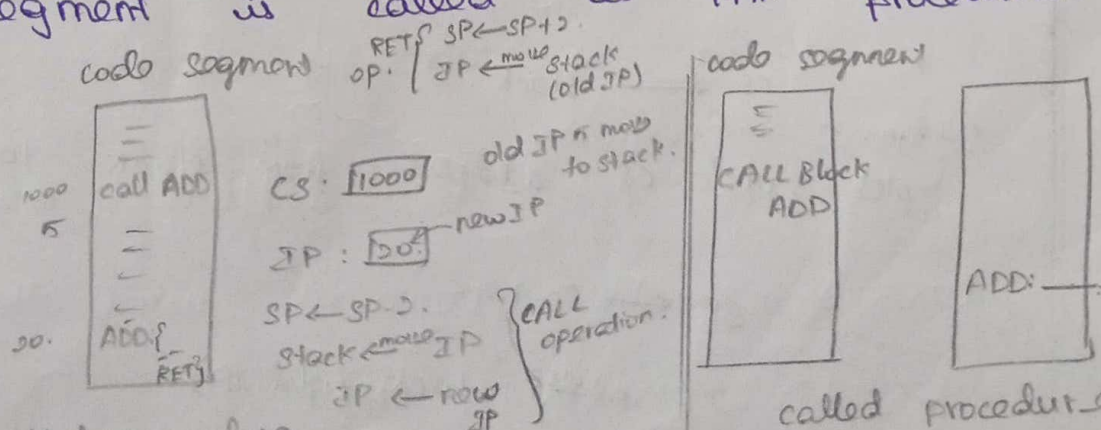
1. NEAR procedure.
2. FAR procedure.

①. NEAR:

If procedure is ~~return~~ written in the same code segment which is calling procedure is called NEAR procedure.

②. FAR:

If procedure is written in different code segment than the calling segment is called as FAR procedure.



start address of segment is stored in CS.

IP - store address of next instructions.

FAR:

① CALL instruction.

* $SP \leftarrow SP - 2$

Stack pointer is decremented by 2.

Stack $\xleftarrow{\text{move}}$ old IP

old IP is move to stack.

* $SP \leftarrow SP - 2$

Stack $\leftarrow$ old CS address.

move old code segment, old, to stack.

* $CS \leftarrow \text{new CS}$

update the CS

* $IP \leftarrow \text{new IP}$

update the IP by new IP.

② RET instruction.

* $SP \leftarrow SP + 2$

last in is CS.

CS $\leftarrow$ stack.

move old CS address from stack to CS.

* $SP \leftarrow SP + 2$

IP $\xleftarrow{\text{move}}$ stack (old IP).

For writing procedure two directives are used.

(just for readability which compiling directives ignored)

1. PROC → indication of start of procedure.

2. ENDP

syntax for procedure:

PROC - NAM PROC → directive to specify name of procedure.

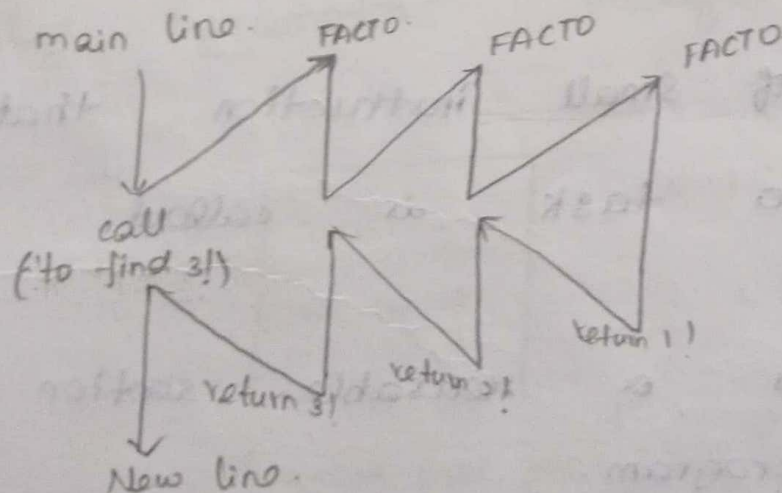
(eg: ADD)

ENDP.

→ Indication of end of procedure.

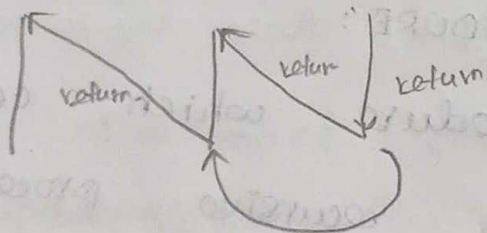
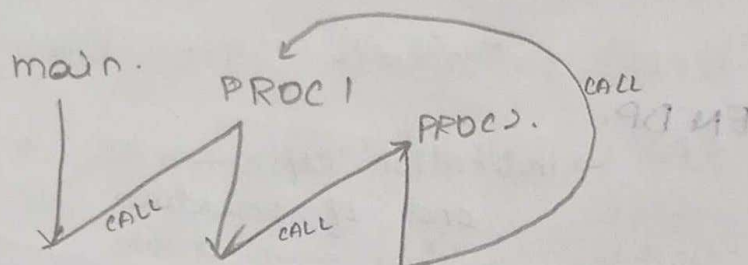
RECURSIVE PROCEDURE:

③. The procedure which call itself is called as recursive procedure.



⑫. RE-ENTERED PROCEDURE.

In some situation procedure 1 called from main, procedure 2 called from procedure 1 and again procedure 1 called from procedure 2. so the program control reenters into procedure 1.



MACRO:

* Group of small instruction that performs a task is called macros.

* It is a reusable section of software program.

* Macro can be defined any where in the program using the directives `MACRO`, `ENDP`

⇒ using this, takes large amount of memory occupation.

* It is used when no. of instructions is less.

* If no. of instructions is more use procedure.

* Additional overhead in calling procedure is CALL, RET, stack used, it takes some amount of time.

PASSING PARAMETERS TO / FROM PROCEDURE:

We can pass parameters in 4 ways.

* through register.

* through dedicated memory location.

* through pointers.

* using stack.

1. Pass parameters through register.

eg:

```
main:
    mov bl, num
    call square
    mov sqx, bl
```

```
square proc NEAR
    mov al, bl
    mul bl
    mov bl, al
    ret
square endp
```

→ it stores data
Data segment
num db 05h
sqx db ?
data ends

Registers.
ax, bx, cx, dx, si, di,
ah, bh, ch, dh

db → defined byte

num → one of memory location label

⇒ num has 05h is move to bl.

⇒ call procedure square

go to procedure

*. mov bl to al, from bl register

(through register passing.

*. mul bl, al * bl = 5 * 5 = 25

*. mov al to sq, 25h

return.

go to main.

⇒ move bl to sq

sq has 25h

⇒ goes to data segment sq db is stored as 25h.

3. Pass Parameters through dedicated memory location.

Main

call square

data segment
num db 05h
sq db ?
data ends

square proc NEAR
mov bl, num
mov al, bl
mul bl
mov sq, al
ret
square endp

main:

⇒ call square.

⇒ go to procedure.

i/p pass through memory in (data segment).

* move num to bl.

(in DS - num define byte 05h)

* move bl to al.

* mul al & bl, $05 \times 05 = 05h$.

* mov al to sqx. (return).

o/p store in DS segment through memory.

3. Pass parameters through pointers:

main.

```
LEA SI, num
LEA DI, sqx
call square.
```

Data segment.

```
num db 05h
sqx db ?
data ends
```

square proc NEAR

mov bl, [SI]

mov al, bl

mul bl

mov [DI], al

ret

square endp

Registers.

al, bl, cl, dl, si, di,

ah, bh, ch, dh

LEA → load effective address.

eg: ⇒ num (1200) this address is loaded to SI.

Now SI has 1200 address that address contain 05h data.

⇒ If `sqr(1201)`, Now `1201` is loaded to `DI`

Now `DI` has `1201` address.

⇒ call `square`.

go to procedure.

* `mov bl, [DI]` → address.

`DI` - has `1200` that has `05h`

Now `05h` move to `bl`.

* move `bl` to `al`

* `mul al & bl`, $05 \times 05 = 05h$.

* `mov [DI], al` → address

`al` value is moved to `DI` address.

(1201)

Now square value is stored in `1201` address.

* then o/p is stored in `DS`.

⇒ Here we use pointer not the registers directly.

4. Pass parameters using stack.

```
main.  
_____  
mov bl, num  
push bx  
call square  
pop bx  
mov sqr, bl  
=====
```

```
Data segment.  
num db 05h  
sqr db ?  
data ends
```

square PROC NEAR

mov BP, SP

mov BX, [BP]

mov al, bl

mul bl

mov [BP], AX

ret
square endp

stack

SP → num

SP-2 → return address.

SP-2 → sqt.

In stack → it push word - not a byte.

⇒

⇒ push BX, 05h is pushed in stack.

⇒ call square.

* SP has address is SP-stack pointer.

moved to BP
data in the

* BP address move to BX.

* result stored in AX (al) moved to BP

address (return)

⇒ pop BX, result 05h is popped from stack.

[BX register's pointer is BP]

⇒ o/p stored in DS.

ROUTINE: (IRS)

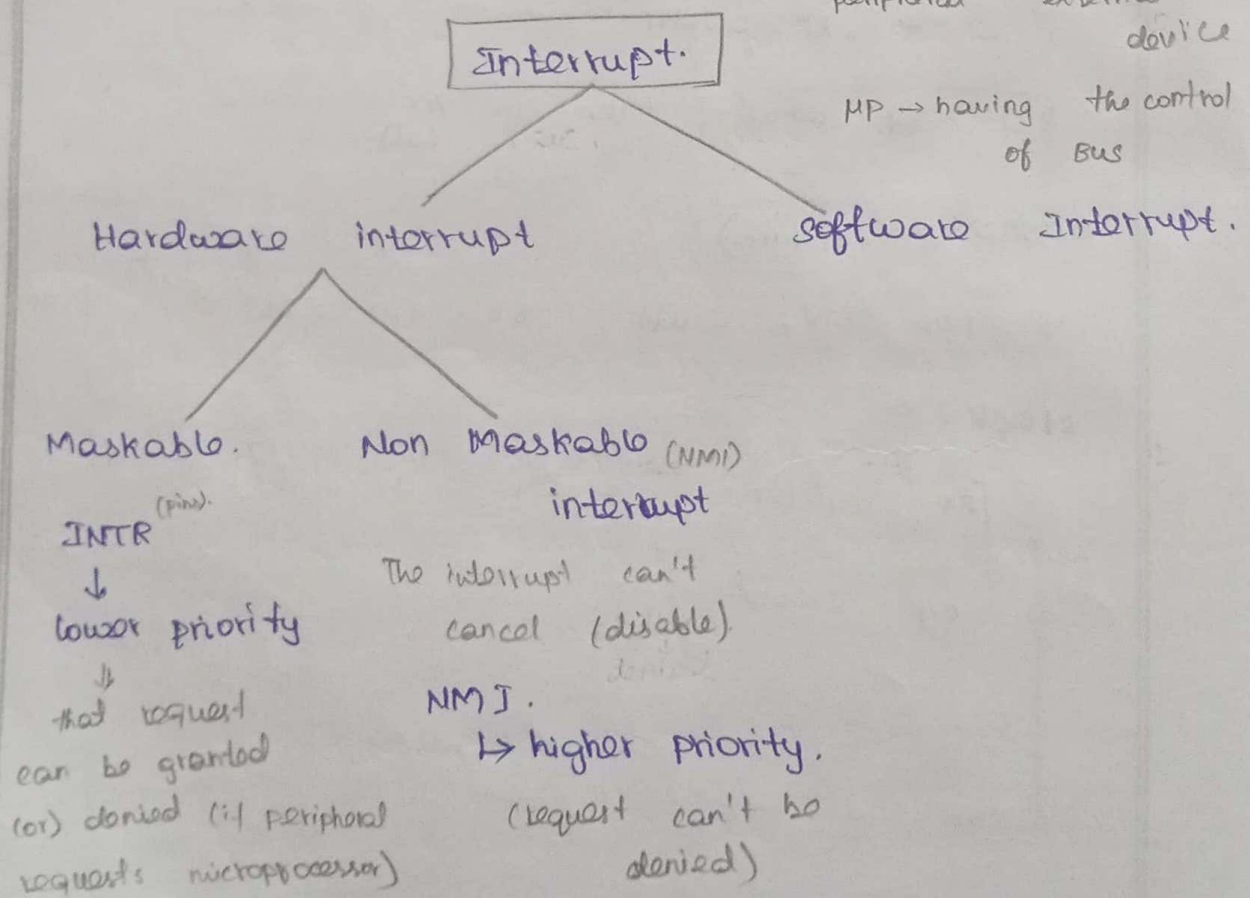
* The method of creating temporary halt during program execution & allows peripheral ^{devices} to access microprocessor is known as interrupt

* The (MP) response to interrupt with ISR which is a short program to instruct microprocessor on how to handle the interrupt.

ISR $\rightarrow$ Interrupt + service routine.

peripheral - external device

MP $\rightarrow$ having the control of bus



* Related to interrupt, we have another pin INTA.

software interrupt.

⇒ has 556 software interrupts.

⇒ INT is the instructions

⇒ Major software interrupts.

Type 0 (INT-0) → divide by zero

Type 1 (INT-1) → single step execution

Type 2 → Non-maskable interrupt

Type 3 → break point interrupt

Type 4 → overflow interrupt.

Vector address: at what address the ISR is stored. (loaded).

The starting address of ISR is called vector address.

Vector table: It has the address of ISR's (information of starting address is available)

ASSEMBLER

DIRECTIVES:

It will help the assembler to understand the assembly language program.

*. Assembler - converting ^{assembly language} high level language into (machine level language program/code) (or) object code.

*. Linker - that will combine/link all the object code and gives as single object code.

*. Loader - that will convert the single linked object code into executable code.

Assembler directives:

* define directive
⇒ dB (define byte)

• It will reserve a byte of memory to a variable.

eg: num db 05h.

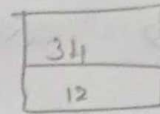
↓
memory location.

1200

⇒ dw (define word)

• It will reserve two bytes of memory to a variable.

eg: num dw 1234h



⇒ dq (define quad)

• It will reserves 8 bytes of memory to a variable.

⇒ dt (define ten bytes)

• It reserves 10 bytes of memory to a variable.

*.ASSUME → used to give a name to logical segment (code (or) data segment).

eg: ASSUME CS : CODE.

ASSUME DS : DATA

*. EVEN → This is used to allocate a next even address to some particular variable. (It make sure that the next address is even)

eg: num db 05h → address 1200

num1 db 06h → address 1201

if:

num db 05h → address 1200

even

num1 db 06h → next even address 1202

* EQU - Equate (assigns value)

eg: EQU TEMP 25

* EXTERN → Extern directive will tell the assembler that the label (or) procedure is defined in some other segment.

eg: EXTERN square procedure name which is in other segment (not in same segment)

* LOCAL → The labels, procedures declared after this local directives will be used only in that module.

eg: LOCAL TEMP 25.

It substitutes only in the present modules, not substitute temp is 25 in other modules.

* GLOBAL → The labels, procedures declared after this GLOBAL directives will be used in other modules also.

* END → end of program.

* ENDS → end of segment

* ENDP → end of procedure.

* PROC → start of procedure.

MODULAR PROGRAMMING:

* The process of dividing the complex program into smaller sub program is known as modular programming.

* Each sub-program is known as module

⇒ [divide & conquer strategy] (uses)

Advantage:

* Speed - up the completion time.

* It will easy to handle.

* Error checking (testing process) will be easy.

* Documentation is easy.

* sub-modules is easy to handle.